

Chapter 4 - Test first, properly

Object-oriented TypeScript, test first

- red-green-refactor, in really small steps
- FizzBuzz, Roman numerals, password rules, leap years

What you will learn

- small steps, and why small is the point
- test names as sentences; arrange / act / assert
- refactoring with the tests green
- `throw new Error(...)` and `assertThrows`
- `assertEquals` vs `assertStrictEquals`
- steps with `t.step` ; helper functions
- katas and "start red" exercises
- doc comment examples versus test files

The cycle

1. **Red** - write **one** test. Watch it fail
2. **Green** - the **least** code that passes
3. **Refactor** - tidy up while everything is green

Why watch it fail? A test you have never seen fail might not test anything at all.

Why so small? Two lines changed, one test red: you know where to look.

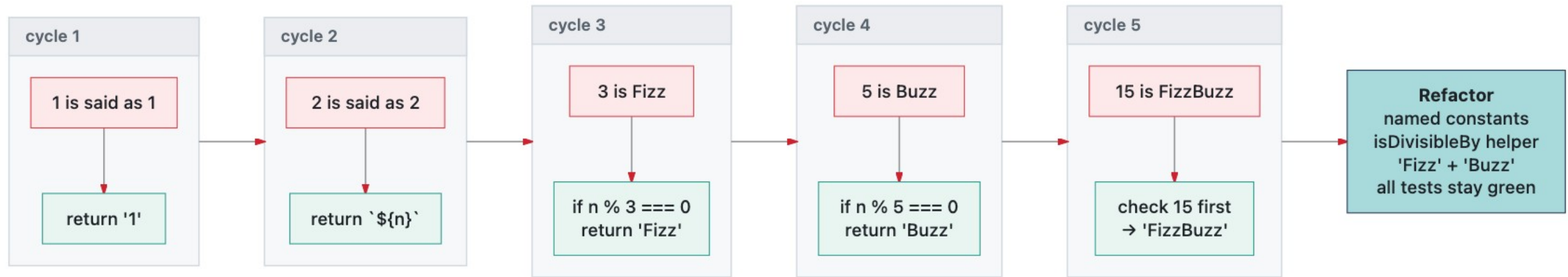
FizzBuzz: the test list first

Count from 1. Multiples of 3: **Fizz**. Of 5: **Buzz**. Of both: **FizzBuzz**.

A plan on paper, simplest first:

- 1 is "1"
- 2 is "2"
- 3 is "Fizz"
- 5 is "Buzz"
- 6 is "Fizz" (and other multiples of 3)
- 15 is "FizzBuzz"

One cycle per line



Red: a new failing test · Green: the smallest change · 6 and 99: already green

Cycle 1

```
Deno.test("1 is said as 1", () => {  
  assertEquals(fizzBuzz(1), "1");  
});
```

```
error: Module not found "src/fizz_buzz.ts".
```

Red. Now the least code that passes:

```
export const fizzBuzz = (n: number): string => "1";
```

Cycle 2: triangulation

```
not ok 2 - 2 is said as 2
-   1
+   2
```

```
export const fizzBuzz = (n: number): string => `${n}`;
```

Generalise only when a second example **forces** you.

Cycles 3 and 4

```
export const fizzBuzz = (n: number): string => {  
  if (n % 3 === 0) {  
    return "Fizz";  
  }  
  if (n % 5 === 0) {  
    return "Buzz";  
  }  
  return `${n}`;  
};
```

"6 and 99 are Fizz" passes straight away: not a cycle, but worth keeping.

Cycle 5: red

1 test failing
ch04_project01_fizzbuzz - 10/3/2026, 3:25:33 PM

5
passed

1
failed

0
skipped

0
type errors

0
lint warnings

Failing tests
✗ 15 is FizzBuzz
tests/fizz_buzz.test.ts:25

```
AssertionError: Values are not equal.  
  
[Diff] Actual / Expected  
  
- Fizz  
+ FizzBuzz  
at tests/fizz_buzz.test.ts:26:3
```

All tests
tests/fizz_buzz.test.ts
✓ 1 is said as 1
✓ 2 is said as 2
✓ 3 is Fizz
✓ 5 is Buzz
✓ other multiples of 3 are Fizz
✗ 15 is FizzBuzz

▶ TAP output

Made by tools/test_report.ts - Markdown version: summary.md

Cycle 5: green

```
export const fizzBuzz = (n: number): string => {  
  if (n % 15 === 0) {  
    return "FizzBuzz";  
  }  
  if (n % 3 === 0) {  
    return "Fizz";  
  }  
  // ... Buzz, then the number  
};
```

It works. **Now** tidy it.

Refactor on green

```
const FIZZ_DIVISOR = 3;
const BUZZ_DIVISOR = 5;

const isDivisibleBy = (n: number, divisor: number): boolean =>
  n % divisor === 0;

export const fizzBuzz = (n: number): string => {
  let answer = "";
  if (isDivisibleBy(n, FIZZ_DIVISOR)) answer += "Fizz";
  if (isDivisibleBy(n, BUZZ_DIVISOR)) answer += "Buzz";
  return answer === "" ? `${n}` : answer;
};
```

(The project writes each `if` with braces.) Ifs the wrong way round?

– BuzzFizz / + FizzBuzz – at once.

Test names are sentences

Not like this	Like this
test1	1 is said as 1
testFizz	3 is Fizz
fizzBuzz works	15 is FizzBuzz
edge case	no answers up to 0

Read the list of tests: you have the rules.

Arrange, act, assert

```
Deno.test("the first five answers, in order", () => {  
  // Arrange  
  const count = 5;  
  
  // Act  
  const answers = fizzBuzzUpTo(count);  
  
  // Assert  
  assertEquals(answers, ["1", "2", "Fizz", "4", "Buzz"]);  
});
```

Act, assert, act, assert? That is two tests.

Examples and tests

Last step, once `fizzBuzz` has settled - a doc comment example:

```
/**
 * What to say for one number: "Fizz", "Buzz", ...
 *
 * @example
 * ```ts
 * import { assertEquals } from "@std/assert";
 *
 * assertEquals(fizzBuzz(3), "Fizz");
 * assertEquals(fizzBuzz(15), "FizzBuzz");
 * assertEquals(fizzBuzz(7), "7");
 * ```
 */
```

Tests **drove** the code · the example **shows** how to use it

Which goes where?

	Doc comment example	Test file
for	someone using it	someone changing it
holds	a few typical calls	every rule and edge
name	none: "lines 14 -22"	a sentence
on failure	the rest is skipped	every test still runs
written	once settled	first, one at a time

Swap the **if** s: tests 7 and 8 say **what** broke, by name; the example only says **where**

Roman numerals

I 1 · V 5 · X 10 · L 50 · C 100 · D 500 · M 1000

- added up, biggest first: VIII is 8
- smaller in front is taken away: IV 4, IX 9, XL 40, CM 900
- 1994 is MCMXCIV

After the units, "10 is X" is red:

– VIIIIII
+ X

The pile of ifs (green!)

```
let result = "";  
let rest = n;  
while (rest >= 10) { result += "X"; rest -= 10; }  
if (rest >= 9) { result += "IX"; rest -= 9; }  
if (rest >= 5) { result += "V"; rest -= 5; }  
if (rest >= 4) { result += "IV"; rest -= 4; }  
while (rest >= 1) { result += "I"; rest -= 1; }  
return result;
```

Five blocks, the same shape. Only the **data** changes.

Data in a table, one loop

```
const NUMERALS: Numeral[] = [  
  { value: 1000, symbol: "M" },  
  { value: 900, symbol: "CM" },  
  // ... down to  
  { value: 1, symbol: "I" },  
];  
  
for (const numeral of NUMERALS) {  
  while (rest >= numeral.value) {  
    result += numeral.symbol;  
    rest -= numeral.value;  
  }  
}
```

A slip, caught

IV typed above V in the table:

```
# Subtest: each symbol on its own
  ok 1 - 1 is I
  not ok 2 - 5 is V
    -   IVI
    +   V
# Subtest: symbols written together are added up
  not ok 3 - 6 is VI
  not ok 4 - 8 is VIII
  not ok 5 - 27 is XXVII
```

Every failure has a 5 in it.

Steps: `t.step`

```
Deno.test("smaller in front is taken away", async (t) => {  
  await t.step("4 is IV", () => {  
    assertEquals(toRoman(4), "IV");  
  });  
  await t.step("9 is IX", () => {  
    assertEquals(toRoman(9), "IX");  
  });  
});
```

- `async (t)`, and `await` before **every** `t.step`
- a failing step does not stop the others

Forget the await

```
not ok 2 - 1 is I
  message: |-
    Didn't complete before parent.
    Await step with `await t.step(...)`.
```

Forget the `async`:

```
TS1308 [ERROR]: 'await' expressions are only allowed within
async functions and at the top levels of modules.
```

A helper makes the steps

```
type Example = { n: number; numeral: string };

const checkExamples = async (
  t: Deno.TestContext, examples: Example[],
): Promise<void> => {
  for (const example of examples) {
    await t.step(`${example.n} is ${example.numeral}`, () => {
      assertEquals(toRoman(example.n), example.numeral);
    });
  }
};
```

A new case is one line of data.

Throwing an error

```
export const minLength = (length: number): Rule => {  
  if (!Number.isInteger(length) || length < 1) {  
    throw new Error(  
      `a minimum length must be a whole number of at least 1, ` +  
      `not ${length}`,  
    );  
  }  
  return { description: `at least ${length} characters`,  
    isMetBy: (password) => password.length >= length };  
};
```

A mistake in the **program**: stop loudly. No checked exceptions,
no `throws` clause.

assertThrows

```
assertThrows(() => minLength(0));
```

Red first: `AssertionError: Expected function to throw.`

Without the `() =>`, `minLength(0)` runs first and its error fails the test:

```
Argument of type 'Rule' is not assignable to parameter  
of type '() => unknown'.
```

Checking the message

```
assertThrows(() => minLength(2.5), Error, "must be a whole number");
```

- the text only has to be **included** in the message
- the wrong text:

Expected error message to include "must be a positive number", but got "a minimum length must be a whole number of at least 1, not 2.5".

A fresh object for every test

```
const makeChecker = (): PasswordChecker =>
  new PasswordChecker([minLength(8), HAS_DIGIT, HAS_UPPER_CASE]);

Deno.test("a password that breaks one rule is not acceptable",
  () => {
    const checker = makeChecker();
    assertEquals(checker.isAcceptable("elephant42"), false);
  });
```

Instead of `@BeforeEach`: nothing leaks between tests, and the setup is in plain sight.

Equal, or the same object?

Equal, or the same object?

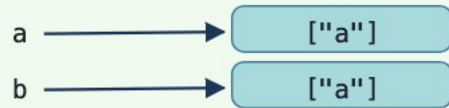
`assertEquals`

same contents?

`assertStrictEquals`

the very same object? (`===`)

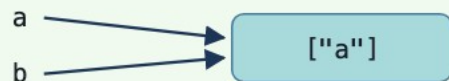
Two objects that look the same



✓ passes

✗ fails

One object with two names



✓ passes

✓ passes

assertStrictEquals

```
const failed = checker.failedRules("Elephants");  
assertStrictEquals(failed[0], HAS_DIGIT);
```

A lookalike copy fails:

```
AssertionError: Values have the same structure but are not  
reference-equal.
```

Surprise: `assertEquals(minLength(8), minLength(8))` **fails** -
two different `isMetBy` functions.

Start red: leap years



Pick one failing test, make it pass, repeat. Never change a test.

Katas, and the habit

A kata: a small problem solved again and again, to practise the **way**.

1. a test list first
2. one test at a time; see it fail
3. the least code
4. refactor on green
5. sentence names; arrange, act, assert
6. edges: 0, empty, one, the boundary, refused input
7. once settled: a doc comment example

JUnit and Deno

JUnit 5	Deno
<code>assertEquals(expected, actual)</code>	<code>assertEquals(actual, expected)</code>
<code>assertSame</code>	<code>assertStrictEquals</code>
<code>assertThrows(X.class, () -> f())</code>	<code>assertThrows(() => f(), Error, "msg")</code>
<code>throw new IllegalArgumentException</code>	<code>throw new Error("...")</code>
<code>@BeforeEach</code>	a helper function
<code>@Nested</code> , <code>@ParameterizedTest</code>	<code>t.step</code> , a table of examples

Challenges

1. Fizz, Buzz, Whizz
2. Make it green (leap years)
3. More password rules
4. Back from Roman
5. The string calculator kata
6. Strict Roman

All test first: see each test fail before you make it pass.